

The diagram illustrates the DCI-Agent workflow. At the top, a 'Query' is input into a search bar. A blue robot character with a question mark above its head and a magnifying glass in its hand represents the agent. An arrow points from the query to the robot. Below the robot, a large pile of colorful document icons represents the 'Corpus (100k-1M Docs)'. An arrow points from the corpus to the robot. To the right of the corpus, a terminal window shows the command '\$ grep PATTERN'. An arrow points from the terminal to the corpus. Below the corpus, a large arrow points down to a list of document snippets. The snippets are numbered 1, 2, ..., 10. The first snippet is 'doc\_307.txt:120:...irrelevant line to config'. The second is 'doc\_997.txt:120:...irrelevant noise'. The third is 'doc\_10223.txt:120:...irrelevant noise'. A bracket on the right side of the list indicates that the output is 'visible to mode'. Below the list, a blue box labeled 'OUTPUT TRUNCATED' is shown. Below this, a dashed line separates the truncated output from the full output. The full output is shown as a list of document snippets, numbered 201, 202, and 400. The first snippet is 'irrelevant noise'. The second is 'doc\_082.jsonl :216: ...core clues....'. The third is 'doc\_094.jsonl :12: ...author core clues....'. At the bottom, a yellow warning triangle with an exclamation mark is shown, followed by the text 'Relevance-agnostic output'.

**RARG**

Corpus

Document-Level Relevance Insertion

Scope Query

Retriever

embed\_recall

**Ranked Scope**

```
/tmp_scope_N.txt
1. doc_317.jsonl
2. doc_082.jsonl
3. doc_941.jsonl
...
10000. doc_2515.jsonl
```

traversal order

```
$ cat /tmp/scope_N.txt |
xargs -d "\n" rg -j1
"PATTERN"
```

matched excerpts raised by doc-level relevance

The diagram illustrates the RARG+ framework architecture, which is designed for relevance-aware retrieval. The process begins with a document collection (e.g., `/tmp/scope_N.txt`) containing documents like `doc_317`, `doc_082`, `doc_094`, and `doc_1992`. These documents are ranked based on relevance, from "More relevant" to "Less relevant". The documents are then split into paragraphs. A "Retriever" module processes these paragraphs to generate a "Ranked paragraph" list. This list is then combined with a "scope query" (e.g., `<qr_paragraph>{top-10 paragraphs}</qr_paragraph>`) to perform "Entry Point Initialization". The final output is a set of retrieved documents, which are then processed by a "Relevance-aware Retrieval" module (represented by a robot icon) to produce the final results.

The diagram illustrates the RARG++ architecture. It starts with an input query and focus: "Instruct: ...", "Query: [scope query]", and "RG focus: [keyword1] [keyword2] ...". These inputs are processed by a "Retriever" (represented by a blue arrow) which performs "match rerank". The retriever outputs a set of documents, some of which are "visible" (highlighted in green) and others "truncated" (highlighted in red). These documents are then processed by a "Match-Level Relevance Insertion" module (represented by a green arrow), which outputs a set of documents, some of which are "visible" (highlighted in green) and others "truncated" (highlighted in red). The final output is the "Final Answer", which includes an "Explanation", the "Answer", and a "Confidence" score (98%).